

Kubernetes Observability Guide: Troubleshooting Microservice Outages with Prometheus & Grafana

1. The Strategic Foundations of Kubernetes Monitoring

In modern Cloud Native environments, the shift from "Monitoring" to "Observability" is a change in investigative philosophy. Traditional monitoring tracks known failure modes—the "known-unknowns." In a volatile Kubernetes cluster, we encounter "unknown-unknowns"—complex, emergent behaviors that occur when hundreds of microservices interact. Observability allows us to infer the internal state of these systems from their external telemetry (metrics, logs, and traces) without shipping new code.

As SREs, we use these signals to maintain **Service Level Objectives (SLOs)**. While a **Service Level Agreement (SLA)** is a contractual promise to customers, our internal SLOs must be stricter. This "reliability buffer" ensures we identify and resolve issues before they breach a contract.

Attribute	Monitoring	Observability
Focus	Known failure modes and symptoms	Unknown failure modes and internal states
Data Model	Pre-defined metrics and thresholds	High-cardinality events and ad-hoc queries
Response	Reactive (alerting on breaches)	Investigative (exploring system behavior)
Goal	Maintain system health	Understand and optimize system behavior

To measure this effectively, we rely on the **Four Golden Signals**. We categorize them by the **RED** method (for services) and the **USE** method (for infrastructure):

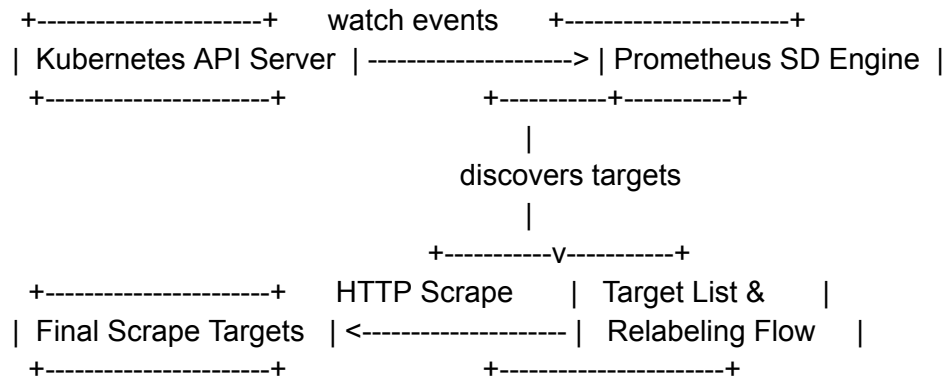
- Latency (RED - Duration): The time taken to serve a request.
- Traffic (RED - Rate): The demand placed on the system (e.g., requests per second).
- Errors (RED/USE - Errors): The rate of requests failing (e.g., HTTP 5xx codes).
- Saturation (USE - Saturation): How "full" your system is.

Senior Lead Nuance: While Latency is the most visible symptom of an outage, **Saturation** is the primary leading indicator of a bottleneck. A resource (CPU/Memory/Disk) that is 100% busy (Utilization) might be fine, but once it starts queuing work (Saturation), latency spikes are inevitable.

2. Architecture & Service Discovery in Kubernetes

Static monitoring configurations are dead in Kubernetes. Because Pods are ephemeral, Prometheus must act as a dynamic participant in the cluster. It addresses container volatility by "watching" the Kubernetes API server for life-cycle events.

The Discovery Flow



Kubernetes SD Roles

Prometheus uses specialized roles to identify telemetry sources:

- Pod: Discovers individual containers. Use Case: Application-specific metrics.
- Service: Discovers Service objects. Use Case: Blackbox probing of service-level availability.
- Endpoints: Discovers pods associated with a service. Use Case: The standard for load-balanced apps.
- EndpointSlice: The modern, more scalable alternative to the endpoints role in large clusters.
- Node: Discovers cluster hosts. Use Case: Infrastructure metrics via node_exporter.
- Ingress: Discovers traffic entry points. Use Case: Monitoring entry latency and ingress controllers.

The Relabeling Flow: Metadata to Meaning

Relabeling is our primary tool for converting raw Kubernetes metadata into actionable production labels. We use the `relabel_configs` block to perform actions like keep (filtering), drop, or labelmap (mapping multiple labels at once).

Example: Mapping the Kubernetes meta label `__meta_kubernetes_pod_label_app` to a clean app label ensures our PromQL remains readable and consistent across environments.

3. Infrastructure as Code: Core Configuration Files

Modern SRE teams use "Monitoring as Code" to eliminate configuration drift.

`prometheus.yml`: Service Discovery & Annotation Logic

We use annotations to give developers control over what gets scraped. This config only scrapes pods where `prometheus.io/scrape` is set to `true`.

scrape_configs:

```
- job_name: 'kubernetes-pods'
```

```
kubernetes_sd_configs:
```

```
- role: pod
```

```
relabel_configs:
```

```
- source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
```

```
  action: keep
```

```
  regex: true
```

```
- action: labelmap
```

```
  regex: __meta_kubernetes_pod_label_(.+)
```

```
- source_labels: [__meta_kubernetes_namespace]
```

```
  target_label: kubernetes_namespace
```

alert_rules.yml: SLO-Based Burn Rate Alerting

Static thresholds like CPU > 80% lead to alert fatigue. Senior SREs prefer **Burn Rate Alerting**, which triggers based on how fast we are exhausting our 30-day "Error Budget."

```
groups:
```

```
- name: availability_alerts
```

```
  rules:
```

```
- alert: HighErrorBudgetBurn
```

```
  # Multi-window burn rate: Alert if 1h burn > 14.4 AND 5m burn > 14.4
```

```
  # This indicates we are on track to exhaust our budget in ~2 days.
```

```
  expr: |
```

```
  (
```

```
    rate(errors_total[1h]) / rate(operations_total[1h]) > (1 - 0.999) * 14.4
```

```
  )
```

```
  and
```

```
  (
```

```
    rate(errors_total[5m]) / rate(operations_total[5m]) > (1 - 0.999) * 14.4
```

```
  )
```

```
  for: 2m
```

```
  labels:
```

```
    severity: page
```

alertmanager.yml: Notification Routing

We route critical alerts to Slack while ensuring placeholders like <YOUR_SLACK_WEBHOOK_URL> are securely managed.

```
route:
```

```
  group_by: ['alertname', 'service']
```

```
  receiver: 'slack-notifications'
```

```
receivers:
```

```
- name: 'slack-notifications'
```

```
  slack_configs:
```

```
    - api_url: '<YOUR_SLACK_WEBHOOK_URL>'
```

```
      channel: '#ops-alerts'
```

```
send_resolved: true
```

```
text: "Summary: {{ .CommonAnnotations.summary }}\nImpact: SLO at risk."
```

grafana.yaml: Dashboard Provisioning

Instead of manual imports, use a dashboard provider to treat visualizations as code.

```
apiVersion: 1
```

```
providers:
```

```
- name: 'SRE-Service-Overview'
```

```
  orgId: 1
```

```
  folder: 'Production'
```

```
  type: file
```

```
  options:
```

```
    path: /etc/grafana/dashboards/service-overview.json
```

4. Advanced Querying with PromQL

PromQL is the engine for root-cause isolation. It allows us to pivot from a symptom to a specific container.

Critical Troubleshooting Queries

- CPU Usage per Container (High Resolution):
sum(node_namespace_pod_container:container_cpu_usage_seconds_total:sum_irate)
by (container)
 - **So What?** We use `irate` over `rate` for microservices because `irate` captures highly volatile, high-resolution spikes. Look for containers hitting their limit; this causes **CPU Throttling**, where the application is "paused" by the kernel, leading to massive latency sawtooth patterns.
- Memory Usage vs. Limit: `container_memory_working_set_bytes / container_spec_memory_limit_bytes`
 - **So What?** If this approaches 1.0 (100%), the container is at risk of an **OOMKill**. Unlike CPU, which throttles, memory is non-compressible; the Linux kernel will terminate the process immediately to protect the host.
- Error Rate Ratio (Errors and Totals Pattern): `rate(errors_total[5m]) / rate(operations_total[5m])`
 - **So What?** We use "Errors and Totals" instead of "Errors and Successes" because it is computationally simpler and handles missing time series better. If a service has zero errors, an `errors / successes` query might fail or return an empty result; `errors / totals` correctly returns 0.

5. Visualization Strategy & Grafana Best Practices

During a 2:00 AM outage, cognitive load is your worst enemy. Dashboards must tell a story from **RED (User Happiness)** to **USE (Machine Happiness)**.

The Dashboard Hierarchy

1. Service Overview (RED): Tracks Rate, Errors, and Duration. If these are red, your users are hurting.
2. Dependency: Monitors downstream health (DBs, third-party APIs). Is a third-party gateway killing your checkout flow?
3. Infrastructure (USE): Monitors Utilization, Saturation, and Errors. Use this to determine if the hardware is the bottleneck.

Best Practices

- Consistent Tagging: Implement a schema for Grafana Annotations. We automate these markers via CI/CD (GitHub Actions). When a release starts, a vertical line appears on all graphs. If a latency spike aligns with that line, you have an immediate root cause.
- Visual Norms: Use color meaningfully (Green = Good, Red = Problem). Normalize axes (e.g., use percentages for CPU) so a dashboard works across nodes with 4 cores and 64 cores without cognitive recalibration.

6. Incident Response: The "Application Down" Framework

When the pager goes off, we follow a structured approach: **Confirm, Communicate, Diagnose, Resolve, Verify**.

Execution Plan (App Down Scenario)

1. Verification: Check the Service Overview dashboard. Are the Golden Signals showing a total failure (100% error rate) or a performance degradation?
2. Communication:
 - **Technical:** Open the incident bridge (Slack/Zoom).
 - **Stakeholders:** Post a non-technical business impact summary (e.g., "Checkout service is currently unable to process Visa payments in the US-East region").
3. Diagnosis: Use PromQL to find the "Why."
 - If Memory Saturation is 100%, check for **OOMKills** in the logs.
 - Check **Annotations** for a deployment. Did a fresh rollout of the auth service cause the spike?
4. Mitigation: SREs prioritize restoration over root-cause fix. Roll back the deployment or scale replicas to absorb a traffic surge.
5. Post-Mortem & Error Budgets: Analyze the impact on the Error Budget. If the SLO was breached, our policy mandates a Release Freeze—suspending new feature work until we've fixed the reliability issues identified in the RCA.

By utilizing the Prometheus/Grafana stack, we transform "blind panic" into a data-driven engineering exercise, ensuring our Kubernetes clusters remain as resilient as the business requires.